ORBIT Assurance Intern Assignment

Latheesh Kumar | 22-05-2026

Vulnerability Scan and Manual Validation

TOOL USED: OWASP ZAP
TARGET APPLICATION: DVWA
ENVIRONMENT: LOCALHOST/DOCKER
OPERATING SYSTEMS: KALI LINUX / WINDOWS 11

OBJECTIVE

The objective of this exercise was to perform a vulnerability scan on a vulnerable web application, identify a security issue, and manually validate the vulnerability to confirm its existence.

ENVIRONMENT SETUP

DVWA was deployed locally localhost:8080.

OWASP ZAP was used to perform automated vulnerability scanning and analysis.

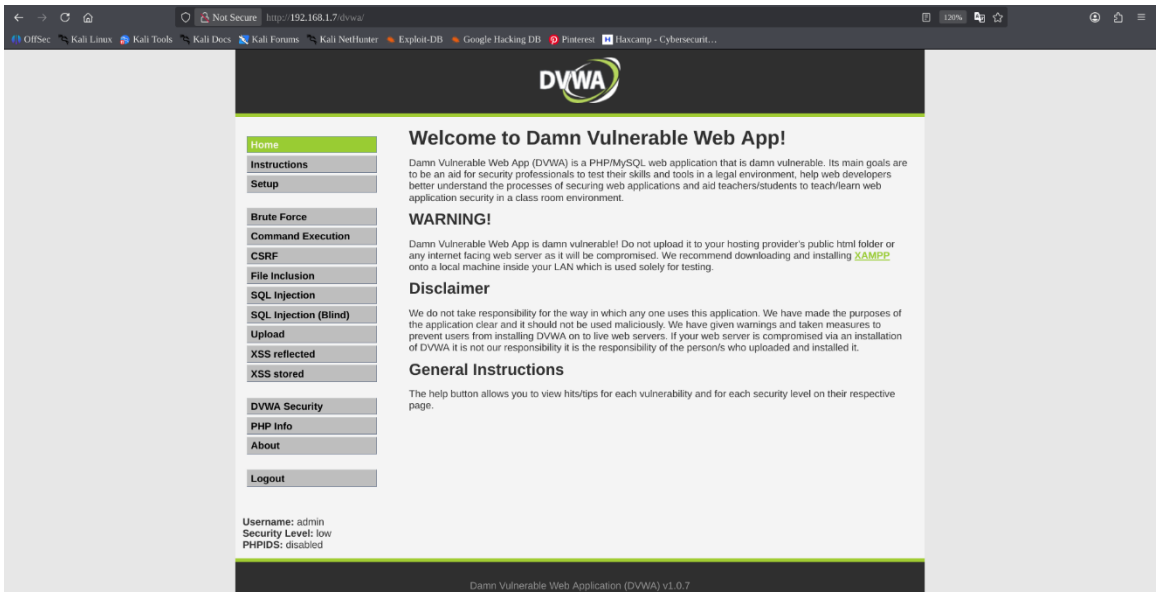


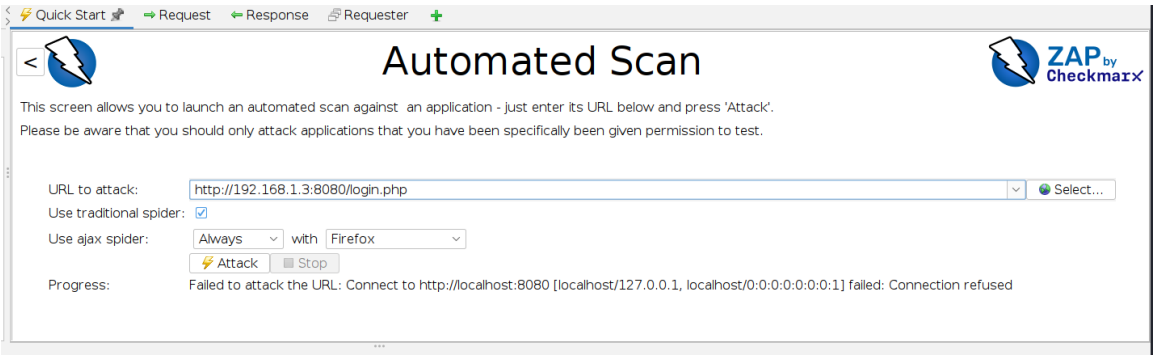
Figure 1: DVWA running locally

SCANNING PROCEDURE

- An OWASP ZAP spider scan was first performed to discover application pages and endpoints. An active scan was then executed to identify vulnerabilities and security misconfigurations.

Steps Performed

1. Opened OWASP ZAP.
2. Added DVWA target URL.
3. Performed Spider Scan.
4. Performed Active Scan.
5. Reviewed generated alerts.



- **Figure 2:** OWASP ZAP active scan against DVWA.

---

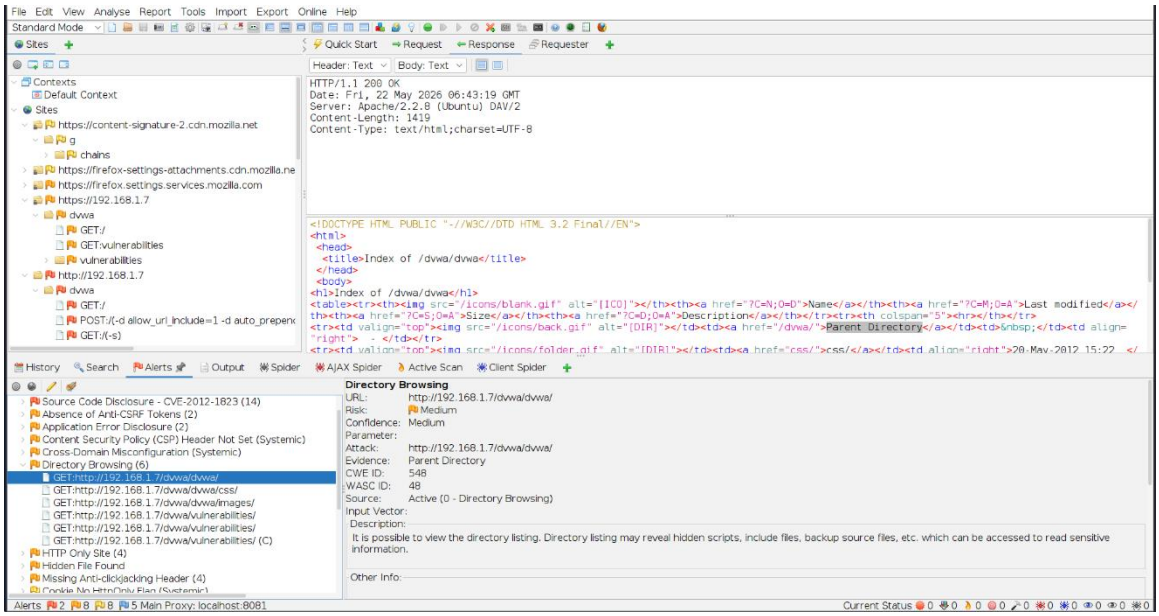
4. Vulnerability Identified

Vulnerability Name

- Directory Browsing

Description

- The scan identified that directory listing was enabled on the web server, allowing users to view directory contents directly through the browser.



- **Figure 3:** OWASP ZAP alert showing Directory Browsing vulnerability.

---

5. Manual Validation

- To manually validate the vulnerability, the exposed directory path was accessed directly through the browser.

Validation Steps

1. Opened the vulnerable directory path manually in the browser.
2. Verified that directory contents were publicly accessible.
3. Confirmed that authentication was not required to access the listing.

URL

http://192.168.1.7/dvwa/dvwa/

http://192.168.1.7/dvwa/dvwa/css/

http://192.168.1.7/dvwa/dvwa/images/

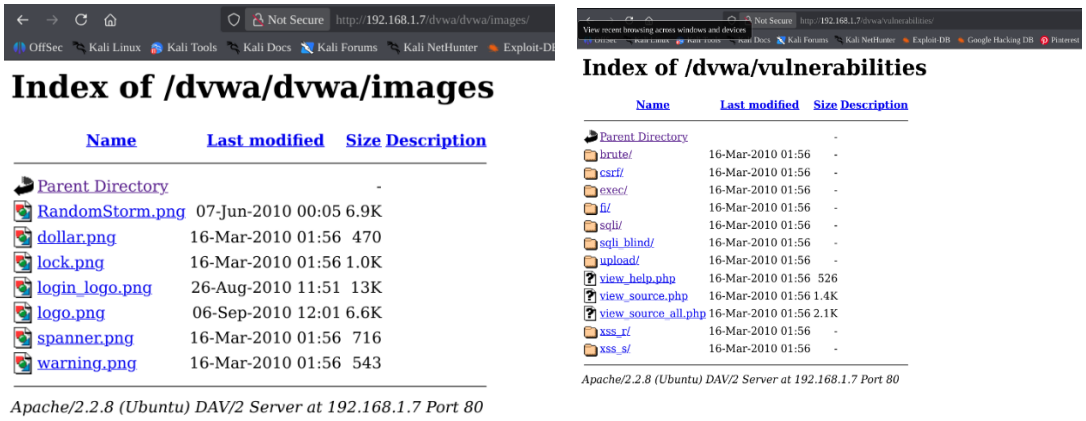
http://192.168.1.7/dvwa/vulnerabilities/

Observation

- The browser displayed:

Index of /uploads

- along with visible files and folders.
- This confirmed that directory browsing was enabled.



- **Figure 4:** Manual validation showing exposed directory listing.

---

6. Security Impact

- Directory browsing can expose sensitive files, application structure, backup files, and configuration data. Attackers may use this information for reconnaissance and further attacks.
- Potential risks include:
- Exposure of sensitive files
- Discovery of application structure
- Enumeration of hidden directories
- Increased attack surface

---

7. Recommended Mitigation

- Directory listing should be disabled in the web server configuration to prevent unauthorized viewing of directory contents.

Apache Mitigation

Options -Indexes

Nginx Mitigation

autoindex off;

- Additional recommendations:
- Restrict access permissions
- Remove unnecessary public directories
- Conduct regular security reviews

---

8. Conclusion

- The vulnerability scan successfully identified a Directory Browsing vulnerability in DVWA. Manual validation confirmed that directory contents were publicly accessible. This exercise demonstrated the importance of combining automated scanning with manual verification for cybersecurity assurance.

---

9. Screenshot Checklist

Figure	Screenshot Required
Figure 1	DVWA running locally
Figure 2	OWASP ZAP active scan
Figure 3	ZAP alert showing Directory Browsing
Figure 4	Browser showing exposed directory listing

PRACTICAL EXERCISE 2

PATCH AND RE-TEST SCENARIO

TOOL USED: OWASP ZAP
TARGET APPLICATION: DVWA
ENVIRONMENT: LOCALHOST/DOCKER
OPERATING SYSTEMS: KALI LINUX / WINDOWS 11

OBJECTIVE

The objective of this exercise was to demonstrate a patch-and-retest scenario by exploiting a vulnerable application before mitigation and verifying that the exploit failed after security controls were applied.

ENVIRONMENT SETUP

DVWA was deployed locally localhost:8080.

OWASP ZAP was used to perform automated vulnerability scanning and analysis.

Vulnerability Name

Command Injection

Description

The Command Injection vulnerability allows attackers to execute system commands through unsanitized user input fields.

---

4. Exploitation Before Mitigation

The DVWA security level was initially configured to:

Low

The vulnerable page used was:

DVWA → Command Injection

Payload Used

127.0.0.1 && whoami

Validation Process

1. Entered the payload into the input field.
2. Submitted the request.
3. Observed command execution results.

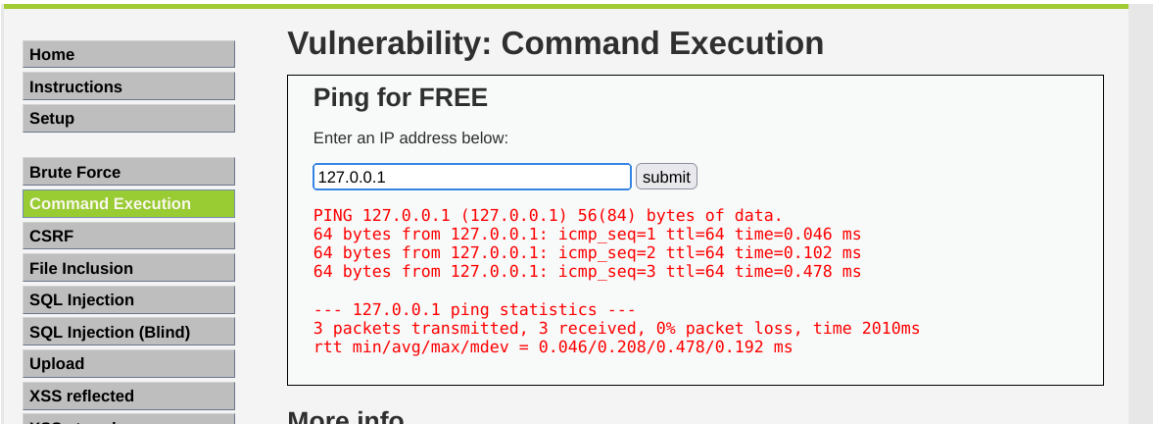
Result

The application executed the additional system command and displayed the current user account.

Example output:

www-data

This confirmed that command injection was successful before mitigation.



- **Figure 1:** Successful command injection before mitigation.

---

5. Mitigation Applied

- To simulate mitigation, the DVWA security level was changed from:

Low → High

- This enabled additional input validation and filtering protections within the application.

- The mitigation simulated:

- Input sanitization
- Application hardening
- Security patching
- Filtering of malicious payloads

- **Figure 2:** DVWA security level changed to High.

---

6. Re-Test After Mitigation

- The same payload was tested again after mitigation.

Payload Re-Tested

127.0.0.1 && whoami

Result

- The application no longer executed the injected command.
- Only normal ping functionality was observed, while the additional command execution failed.
- This confirmed that the mitigation was effective.

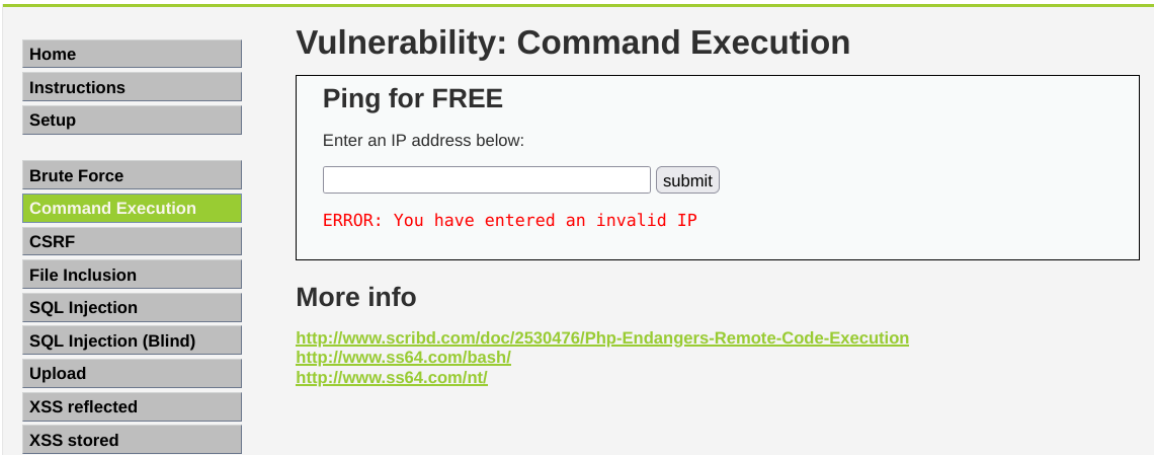


Figure 3: Command injection blocked after mitigation.

---

7. Results Comparison

Phase	Result
Before Mitigation	Command executed successfully
After Mitigation	Command execution blocked

---

8. Security Impact

- Command Injection vulnerabilities may allow attackers to:
  - Execute arbitrary system commands
  - Access sensitive files
  - Escalate privileges
  - Install malware
  - Fully compromise the server
- This type of vulnerability is considered highly critical because it may lead to complete system compromise.

---

9. Conclusion

- The exploit was successfully demonstrated before mitigation and failed after the security control was applied. This confirmed that the mitigation effectively reduced the vulnerability and validated the security assurance process.

---

10. Screenshot Checklist

Figure	Screenshot Required
Figure 1	Successful exploit before mitigation
Figure 2	Security level changed to High
Figure 3	Exploit blocked after mitigation

Practical Exercise 4

Python Script to Validate a Security Control

1. Objective

- The objective of this exercise was to create a Python script that validates a security control by checking whether SSH port 22 is closed on a target system and displaying a PASS or FAIL result.

---

2. Environment Setup

- The script was executed in a Linux environment using Python 3.

Tools Used

- Python 3
- Linux Terminal
- Socket Library

---

3. Python Validation Script

- The following Python script was created to verify whether SSH port 22 was accessible on the target system.

```
import socket

HOST = "192.168.1.10"
PORT = 22
TIMEOUT = 3

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(TIMEOUT)

try:
   s.connect((HOST, PORT))
   s.close()
   print("FAIL: SSH port 22 is OPEN")

except:
   print("PASS: SSH port 22 is CLOSED")
```

4. Script Explanation

- The script:

1. Creates a TCP socket connection.
2. Attempts to connect to port 22 on the target system.
3. If the connection succeeds:
    - The port is open.
    - The validation result is FAIL.
4. If the connection fails:
    - The port is closed.
    - The validation result is PASS.

```python
import socket

# Target details
host = input("Enter IP address: ")
port = int(input("Enter port number: "))

# Create socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(2)

# Try connecting to the port
result = s.connect_ex((host, port))

# Validation
if result == 0:
    print("PASS - Port is closed")
else:
    print("FAIL - Port is open")

# Close socket
s.close()

```

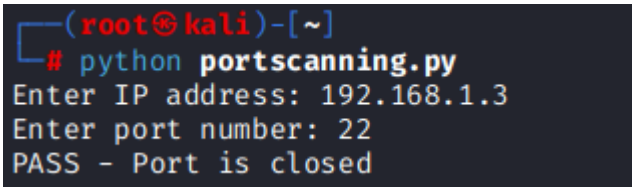
5. Validation Result

Example PASS Output

PASS: SSH port 22 is CLOSED

Example FAIL Output

FAIL: SSH port 22 is OPEN



---

6. Security Importance

- This type of validation script helps security teams:
- continuously verify security controls,
- automate configuration checks,
- identify unauthorized changes,
- support continuous assurance activities.
- Automated validation improves consistency and reduces manual effort in cybersecurity operations.

---

7. Conclusion

- The Python script successfully validated the SSH security control by checking whether port 22 was closed and generating a clear PASS/FAIL result. This demonstrates how automation can support continuous security assurance.

---

8. Screenshot Checklist

Figure	Screenshot Required
Figure 1	Python script in terminal/editor
Figure 2	PASS or FAIL execution result

Practical Exercise 5 — ORBIT Scenario Simulation

---

1. Objective

The objective of this exercise was to simulate an ORBIT security assurance scenario by disabling root SSH login, attempting access, and verifying that the mitigation successfully prevented unauthorized administrative access.

---

2. Environment Setup

The exercise was performed in a Linux test environment with SSH enabled.

Tools Used

- Linux Server
- OpenSSH
- Terminal
- SSH Client

---

3. ORBIT Scenario Overview

This exercise simulated a common hardening control used in enterprise environments.

The objective was to:

1. Disable direct root SSH access.
2. Attempt root login.
3. Verify that the mitigation blocked the login attempt.

This continuous assurance and security control validation.

---

4. Mitigation Applied

The SSH configuration demonstrates file was modified:

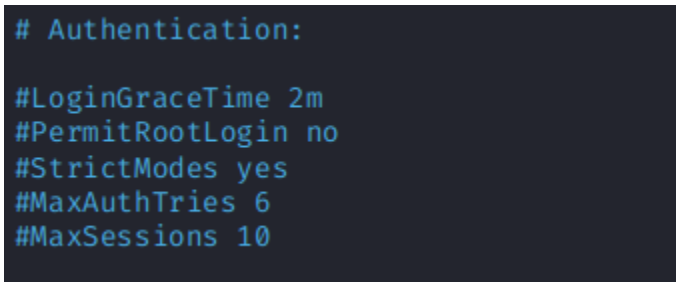
- sudo nano /etc/ssh/sshd_config

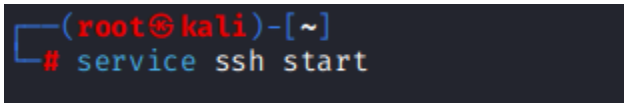
The following setting was configured:

- PermitRootLogin no

The SSH service was then restarted:

- sudo systemctl restart ssh





---

5. Validation Attempt

After applying the mitigation, a root SSH login attempt was performed using:

- ssh root@<target-ip>

---

6. Validation Result

The login attempt failed and the system denied direct root access.

```
C:\Users\menak>ssh root@192.168.1.8
root@192.168.1.8's password:
Permission denied, please try again.
root@192.168.1.8's password:
Permission denied, please try again.
root@192.168.1.8's password:
root@192.168.1.8: Permission denied (publickey,password).
```

Example output:

- Permission denied

This confirmed that:

- root SSH login was successfully disabled,
- the mitigation was functioning correctly,
- unauthorized administrative access was prevented.

---

7. Security Impact

Disabling root SSH login helps:

- reduce brute-force attack exposure,
- prevent direct privileged access,
- improve accountability through individual user accounts,
- strengthen server hardening.

This is considered a common Linux security best practice.

---

8. Conclusion

The ORBIT scenario was successfully simulated by disabling root SSH login and validating that direct root access was blocked. The mitigation worked as expected and demonstrated effective security control validation.

---

9. Screenshot Checklist

Figure Screenshot Required

Figure 1 SSH configuration showing PermitRootLogin no

Figure 2 SSH service restart

Figure 3 Failed root SSH login attempt

Practical Exercise -5

Simulated Attack Detection Using Wazuh

---

1. Objective

The objective of this exercise was to simulate failed authentication attempts on a monitored Windows endpoint and verify that Wazuh successfully detected and logged the activity.

---

2. Environment Setup

The exercise was performed in a virtual lab environment consisting of:

System	Purpose
Kali Linux VM	Wazuh Manager and Dashboard
Windows 10 VM	Monitored endpoint with Wazuh Agent

---

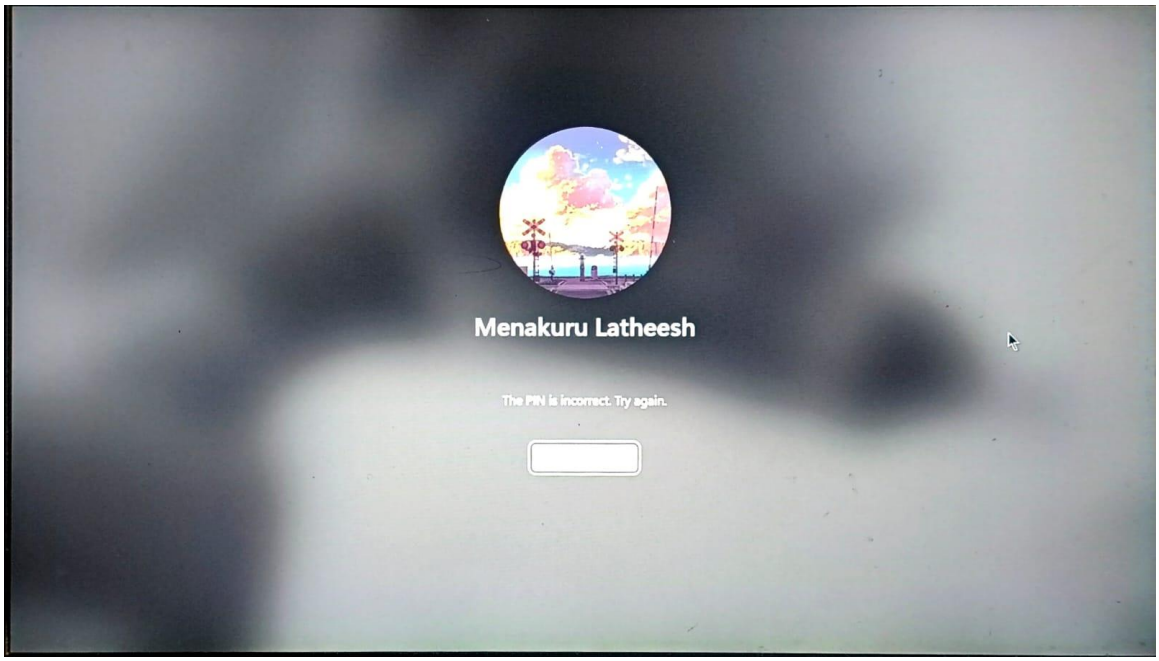
3. Tools Used

- Wazuh
- Windows 11
- Kali Linux
- Windows Event Logs
- VirtualBox

4. Attack Simulation

Failed login attempts were intentionally generated on the Windows 11 by entering incorrect credentials multiple times.

The objective was to simulate authentication failure activity commonly associated with brute-force attacks and unauthorized access attempts.



5. Wazuh Detection Process

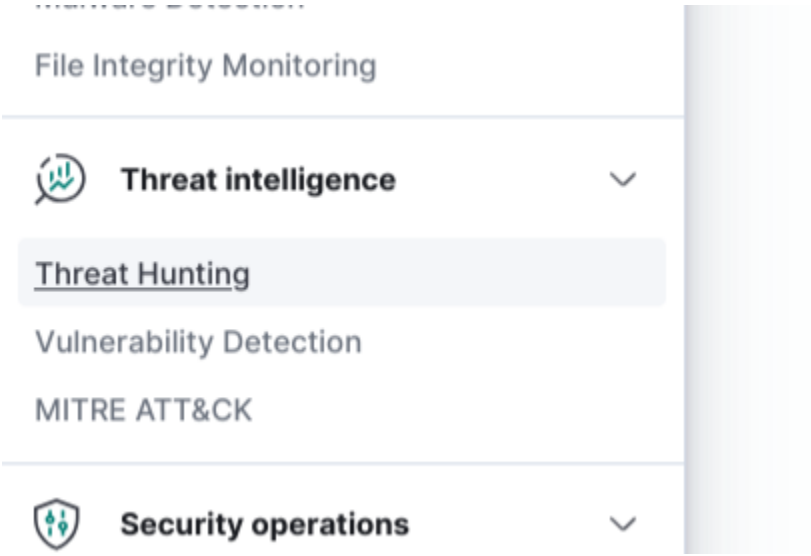
The Windows endpoint was configured with the Wazuh agent, which forwarded Windows Security Event logs to the Wazuh manager hosted on the Kali Linux virtual machine.

The following steps were performed:

1. Generated multiple failed login attempts on the Windows VM.
2. Opened the Wazuh dashboard on Kali Linux.
3. Navigated to:

- Threat Hunting → Events

4. Searched for authentication-related alerts and failed login events.



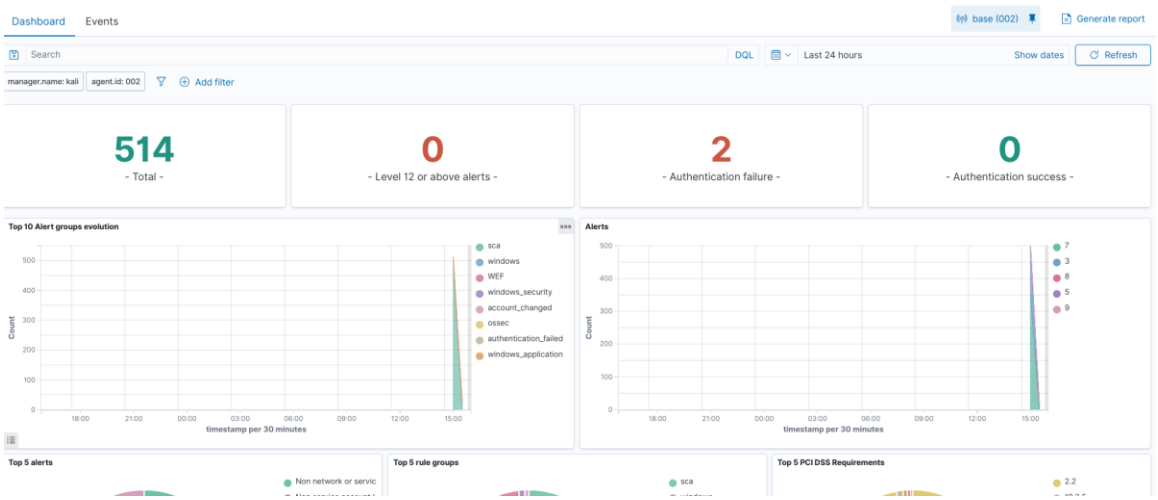
6. Detection Results

Wazuh successfully detected and logged the failed authentication attempts.

The dashboard displayed:

- authentication failure alerts,
- event timestamps,
- monitored endpoint details,
- Windows security event information.

The alerts confirmed that Wazuh was successfully monitoring authentication-related activity from the Windows endpoint.



7. Event Information

The failed login attempts generated Windows Security Event:

- Event ID 4625

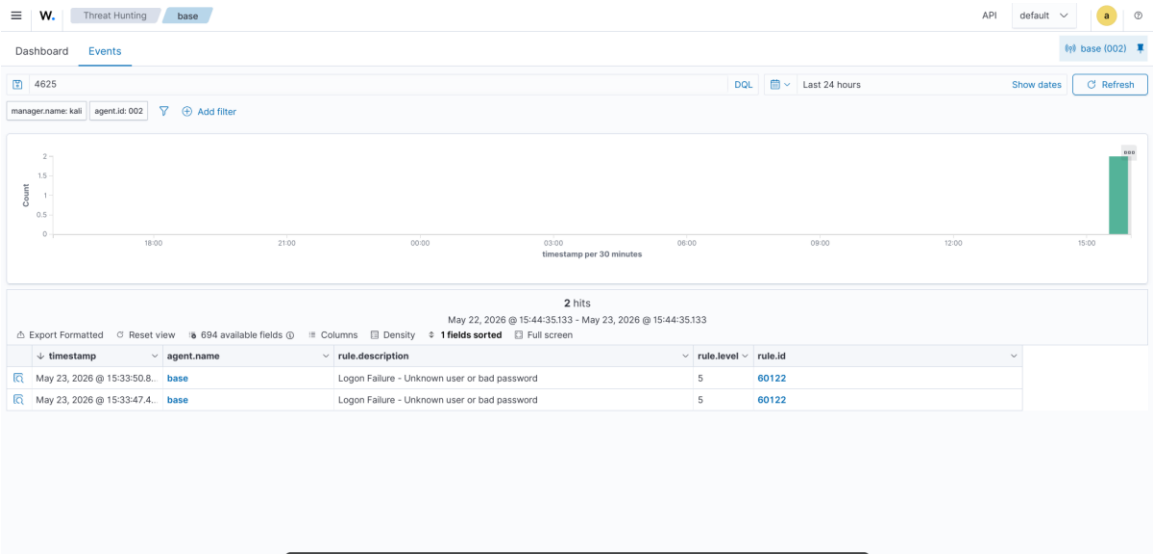
which represents:

- Failed Logon Attempt

This event is commonly monitored by SIEM platforms to identify:

- brute-force attacks,

- password guessing,
- unauthorized access attempts.



---

8. Security Importance

Monitoring authentication failures is important because repeated failed login attempts may indicate:

- brute-force attacks,
- credential stuffing,
- unauthorized access attempts,
- account enumeration activity.

SIEM solutions such as Wazuh help security teams identify and respond to suspicious authentication behavior in real time.

---

9. Conclusion

The simulated failed login activity was successfully detected and logged by Wazuh through centralized monitoring and event analysis. The exercise demonstrated the effectiveness of SIEM-based authentication monitoring within a controlled lab environment.

---

10. Screenshot Checklist

Figure	Screenshot Required
Figure 1	Failed login attempts on Windows VM
Figure 2	Wazuh Threat Hunting dashboard
Figure 3	Authentication failure alerts
Figure 4	Event details showing Event ID 4625

END